



Universidad Tecnológica de Durango

Tecnologías de la Información

Aplicaciones Web

Proyecto final

“Aplicaciones web”

Alumnos:

- Barraza Torres Jesús Daniel

4°A BIS

Docente:

- Ing. Juan Pablo Ríos Martínez

Abril 2026

Tabla de Ilustraciones

Ilustración 1. Carpeta del proyecto.....	5
Ilustración 2. Carpeta publi.c	5
Ilustración 3. Tabla de flujo de aplicación.....	6
Ilustración 4. Diagrama Entidad-Relación de la página	7
Ilustración 5. Sesión	8
Ilustración 6. Registro del usuario.....	8
Ilustración 7. Logout.....	9
Ilustración 8. Login	9
Ilustración 9. función de búsqueda total de reserva	10
Ilustración 10. Función de búsqueda definida de la tabla.....	10
Ilustración 11. Función de borrar una reserva.....	11
Ilustración 12. Función de modificar una reserva	11
Ilustración 13. Función de agregar una reserva	11
Ilustración 14. Función de la página principalE.....	12
Ilustración 15. Página de inicio	13
Ilustración 16. Tabla de reservaciones.....	13
Ilustración 17. submitDelete().....	14
Ilustración 18. LoginRaR()	15

Tabla de contenido

Descripción del sistema	4
Nombre	4
Descripción	4
Estructura del sistema	5
Organización de carpetas	5
Descripción del flujo de aplicación	6
Base de datos	7
Diagrama simple.....	7
Campos principales.....	7
Funciones principales	8
Página de inicio	8
Funciones de la lista.....	10
Otro (importante)	12
Estilos y diseños	13
Motivos del diseño.....	13
Relación diseño-componente	13
Peticiones web.....	14
Link del git.....	16

Descripción del sistema

Nombre

- Página web de un sistema de “citas” para salas de un estudio de grabación

Descripción

- Programa que permite a los operadores de manera remota apartar salas individuales para la grabación de grupos

Estructura del sistema

Organización de carpetas

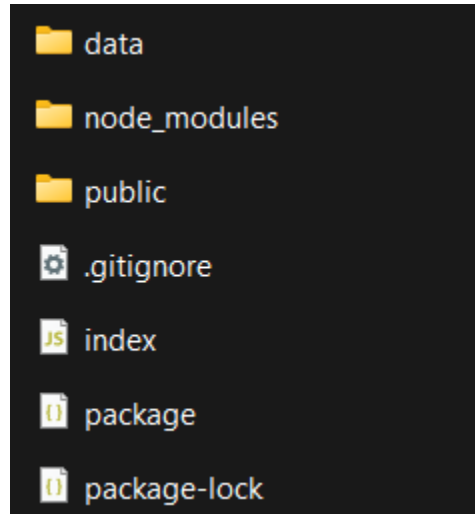


Ilustración 1. Carpeta del proyecto

Dentro de la carpeta del proyecto se puede observar, aparte de archivos para el funcionamiento general del proyecto (los packages, el index, el archivo .gitignore para github) 3 carpetas: data, node_modules y public.

La carpeta data aloja específicamente todo con respecto a la base de datos, siendo el módulo para activarlo y el archivo sql del mismo. Dentro de node_modules se encuentra todo lo que corresponde a funciones del programa. Dentro de public se encuentran archivos y carpetas.



Ilustración 2. Carpeta publi.c

Dentro de esta carpeta podemos observar las páginas de html para el inicio de sesión por parte del staff y de la tabla de reservas, junto a su aspecto (la carpeta css) y su funcionalidad general (carpeta js).

Descripción del flujo de aplicación

La aplicación web es, por un lado, lo suficientemente simple para explicar en puras palabras.

La página del inicio te permite registrarte para que la base de datos pueda mantener una sesión mediante correo electrónico o número telefónico; también permite iniciar sesión para que, con las credenciales, entres directamente hacia la página de las reservas.

Dentro de la página de reservas se puede crear, eliminar, modificar y visualizar, dentro de una forma de lista seleccionable, todas las reservaciones que se están realizando por el momento. Para poder volver a la página de inicio de sesión, puedes cerrar sesión.

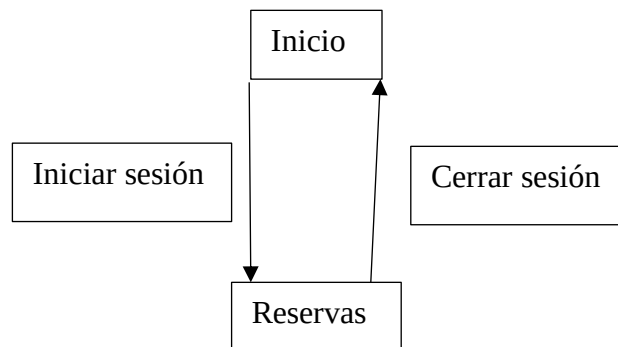


Ilustración 3. Tabla de flujo de aplicación

Base de datos

Diagrama simple

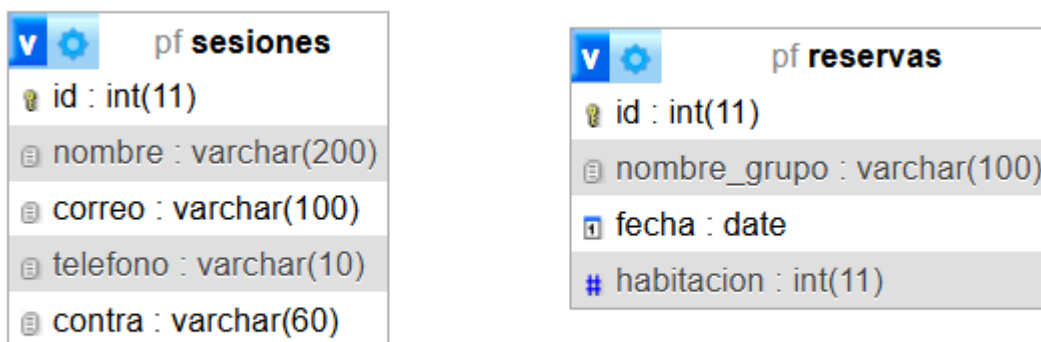


Ilustración 4. Diagrama Entidad-Relación de la página

Campos principales

Podemos observar el diagrama de la base de datos de la aplicación web, este tiene claramente 2 tablas los cuales, dado a las claves de inicio de sesión no están relacionadas entre sí.

Por un lado, tenemos la tabla de sesiones, la cual posee los datos del nombre, correo, teléfono, y contraseña, los cuales son partes esenciales para la identificación e inicio de sesión.

Por el otro lado tenemos las tablas de reservas los cuales son los nombre_grupo, fecha y habitación, que sirve para ver quién hizo la reserva, cuándo será y en donde será.

Funciones principales

```
//sesión
app.use(session({
  secret: "secreto",
  resave: false,
  saveUninitialized: true,
}));
```

Ilustración 5. Sesión

Tras el inicio de la aplicación, tienes la posibilidad de realizar una sesión la cual se guarda mediante la función de “express-session”, este guarda dentro de una “cookie” las credenciales del inicio de sesión para el posterior cambio de página.

[Página de inicio](#)

```
//Registro
app.post("/usuarios", async (req, res) => {
  try {
    const {nombre, correo, telefono, contra} = req.body
    const contraHash = await bcrypt.hash(contra, 10);
    const sql = "INSERT INTO sesiones (nombre, correo, telefono, contra) VALUES (?, ?, ?, ?)"
    db.query(sql, [nombre, correo, telefono, contraHash], (err, results) => {
      if (err) {
        return res.status(500).send(err)
      } else {
        res.send(results)
      }
    })
  } catch (err) {
    res.status(500).send(err)
  }
});
```

Ilustración 6. Registro del usuario


```
//Inicio de sesión
app.post("/login", async (req, res) => {
  const {correo, telefono, contra} = req.body
  const sql = "SELECT * FROM sesiones WHERE correo = ? OR telefono = ?"
  db.query(sql, [correo, telefono], async (err, results) => {
    if (results.length == 0) {
      return res.status(401).send("Usuario no encontrado")
    } else {
      const usuario = results[0]
      const ver = await bcrypt.compare(contra, usuario.contra)
      if (ver) {
        req.session.usuario = {
          id: usuario.id,
          nombre: usuario.nombre,
        }
        res.send("Inicio de sesión exitoso")
      } else {
        return res.status(400).send("Contraseña incorrecta")
      }
    }
  });
});
});
```

Ilustración 8. Login

```
//Cierre de sesión
app.get("/logout", (req, res) => {
  req.session.destroy();
  res.send("Sesión cerrada")
});
```

Ilustración 7. Logout

En las siguientes ilustraciones tenemos las 3 funciones fundamentales del inicio de sesión: Inicio de sesión, registro, y Cierre de sesión. Estas distribuciones de funciones se dan por:

- Registro: Función POST
- Login: Función GET
- Logout: Función GET

Registro ingresa el nombre, correo, teléfono y contraseña a la tabla de datos, mientras que el login verifica si existe el correo/número telefónico y la contraseña coincide para pasar

Aplicaciones web

Página | 9

a la siguiente página y logout simplemente destruye la sesión y vuelve a la página de inicio de sesión.

Funciones de la lista

```
//Operaciones CRUD (punto 4)
//Selección completa de datos
app.get("/reserva", (req, res) => {
  db.query("SELECT * FROM reservas", (err, results) => {
    if (err) {
      return res.status(500).send(err);
    }
    res.json(results)
  });
});
```

Ilustración 9. función de búsqueda total de reserva

```
//Selección parcial de datos
app.get("/reserva/:nombre_grupo", (req, res) => {
  const {nombre_grupo} = req.params;
  const sql = "SELECT * FROM reservas WHERE nombre_grupo LIKE ?"
  db.query(sql, [`%${nombre_grupo}%`], (err, results) => {
    if (err) {
      return res.status(500).send(err)
    } else {
      res.json(results)
    }
  });
});
```

Ilustración 10. Función de búsqueda definida de la tabla

```
//Añadido de datos
app.post("/reserva/agregar", (req, res) => {
  const {nombre_grupo, fecha, habitacion} = req.body;
  const sql = "INSERT INTO reservas (nombre_grupo, fecha, habitacion) VALUES (?, ?, ?)";
  db.query(sql, [nombre_grupo, fecha, habitacion], (err, result) => {
    if (err) {
      return res.status(500).send(err);
    }
    res.send("usuario creado", result)
  });
});
```

Ilustración 13. Función de agregar una reserva

```
//Actualización de datos
app.put("/reserva/:id", (req, res) => {
  const {id} = req.params;
  const {nombre_grupo, fecha, habitacion} = req.body
  const sql = "UPDATE reservas SET nombre_grupo = ?, fecha = ?, habitacion = ? WHERE id = ?"
  db.query(sql, [nombre_grupo, fecha, habitacion, id], (err, result) => {
    if (err) {
      return res.status(500).send(err);
    } else {
      res.send("usuario actualizado", result);
    }
  });
});
```

Ilustración 12. Función de modificar una reserva

```
//Eliminación de datos
app.delete("/reserva/:id", (req, res) => {
  const {id} = req.params
  const sql = "DELETE FROM reservas WHERE id = ?"
  db.query(sql, [id], (err, result) => {
    if (err) {
      return res.status(500).send(err);
    } else {
      res.send("usuario actualizado")
    }
  });
});
```

Ilustración 11. Función de borrar una reserva

En todas las ilustraciones de arriba podemos observar todas las funciones de la tabla, las cuales son:

- Mostrar: GET
- Buscar: GET
- Agregar: POST
- Modificar: PUT
- Eliminar: DELETE

Cada una de las funciones siguen con sus cosas en cuestión, siendo el de mostrar el que se da cuando la barra de búsqueda no tiene nada en él, mostrando todo.

Luego sería el de buscar (o mostrar grupo) donde, buscando el nombre del grupo, muestra dentro de las palabras aquellas cosas que tienen ese tipo de datos.

Para agregar es simplemente una función donde agrega los 3 apartados de la tabla dentro de la base de datos.

La modificar es, para cuando seleccionas un grupo de la tabla, te da la chance de cambiar los apartados de la tabla e insertarla en la base de datos.

La función eliminar es para... Eliminar, mediante la selección de la tabla.

Otro (importante)

```
//Apertura de la página web (si...)
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, './public/inicio.html'));
});
```

Ilustración 14. Función de la página principal

En esta ilustración podemos observar lo más importante para que la página funcione: Cuando este tiene una dirección normal ("/"), esta muestra la página de inicio, la cual se encuentra en la carpeta "public". Esto permite el iniciar un servicio de react con páginas html (que por algún motivo de normal que hacerlo con un archivo JS o JSX).

Estilos y diseños

Motivos del diseño

Uhh... Romperé mi profesionalismo para ser honesto, este diseño fue rápido y poco específico, ya que este no es un proyecto 100% de mi interés. Pero bueno, simplemente quedémonos con las especificaciones comunes de todo lo que se daba.

Bienvenido



Ilustración 15. Página de inicio

En esta ilustración podemos observar mi flojera a la hora de hacer diseños cuando el proyecto no es 100% importante. Decidí hacer un sistema funcional y simple sobre un sistema bonito y visualmente apelante.

Si tuviera que trabajar esto el diseño puede ser 100% diferente

Relación diseño-componente

El diseño intenta ser lo más simplista, así que la operación se basa únicamente en términos de la lectura.

Buscar por nombre de grupo Buscar			
ID	Grupo	Fecha	Habitación
3	Mi casa	2026-04-18T06:00:00.000Z	1
Agregar reserva Actualizar reserva Eliminar reserva			

Ilustración 16. Tabla de reservaciones

En esta ilustración podemos observar lo simple de la organización de este programa. Simplemente es una barra de búsqueda junto con una tabla que muestra datos y 3 botones.

Peticiones web

Las peticiones son partes del frontend para relacionar cosas del backend, los cuales se dan mediante las solicitudes de los hipervínculos que se mostraron 2 capítulos atrás. Las solicitudes se realizan mediante una función asíncrona, donde la solicitud envía una promesa mientras se continúan con los procesos hasta que se pueda realizar la solicitud.

A continuación se mostrarán unos ejemplos dentro de la página:

```
submitDelete.addEventListener("click", async () => {  
  const id = document.getElementById("delete_id").value;  
  try {  
    const res = await fetch(`/reserva/${id}`, { method: "DELETE" });  
    const data = await res.text();  
    showMessage("Eliminar", data);  
    popupEliminar.style.display = "none";  
    cargarReservas();  
  } catch (err) {  
    console.error(err);  
    showMessage("Error", "No se pudo eliminar la reserva");  
  }  
});
```

Ilustración 17. submitDelete()

En esta ilustración podemos observar la función de eliminar como ejemplo de explicación. Aquí este pide la ID del elemento para poder realizar la función para luego, mediante la función de “await”, pedir la solicitud de DELETE para borrar la tabla. A partir del texto de resultado (con otro await), este intenta borrar. Si en algún momento se da un error, este lo agarra mostrando el error dentro de la consola.

```
// Login
const LoginRaR = document.getElementById("loginbtn");
LoginRaR.addEventListener("click", async () => {
  const ident = document.getElementById("ident").value;
  const contra = document.getElementById("contra").value;

  try {
    const res = await fetch("/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ correo: ident, telefono: ident, contra })
    });

    if (res.ok) {
      window.location.href = "/reservas.html";
    } else {
      const data = await res.text();
      showMessage("Error", data);
    }
  } catch (err) {
    console.error(err);
    showMessage("Error", "Error al iniciar sesión");
  }
});
```

Ilustración 18. LoginRaR()

En esta ilustración podemos observar la función de Login, así que toca explicar. Tras pulsar el botón, este lee los datos dentro de “ident” y “contra” (el identificador (correo o teléfono) y la contraseña) y los saca como variables, de ahí se intenta obtener la función de login POST enviando como identificador el correo/teléfono y la contraseña. En caso de que salga exitoso este te lleva a la página de reservas, si no entonces te muestra el error dentro del resultado. En caso de que se muestre un error desconocido, este te muestra el mensaje del error al iniciar sesión.

Link del git

[GitHub - Jesus-Barraza/Paginas_web: Materia de páginas web · GitHub](#)

En este link se encuentra la página de páginas web, para poder ver el proyecto, entra en la carpeta de PF.

Honestamente ya se puede descansar, son las 9:30 pm y ando haciendo este documento porque tenía el código completo antes del documento :skull:

Espero que esto no quede muy confuso...